Interactive Fourier Epicycle Drawing Simulator

Final Project Proposal

Project Description

This project will develop an interactive application that reconstructs user-drawn curves as a chain of rotating epicycles. The application will open with a blank drawing canvas on which the user can sketch a path. After which, the program will sample and normalize the path, [1] compute its Fourier coefficients, and open a visualization of the curve being traced by the endpoint of a single chain of rotating vectors. Users will also be able to choose from several preset drawings. The exact method for creating presets is still not entirely clear but I plan on reverse engineering the process from the fourth video of Grant Sanderson's (3Blue1Brown) series on differential equations. [2]

Unlike some visualizations that decompose the x and y components separately, I plan on describing both components as a single point in the complex plane, $(x + yi)$, allowing the entire curve to be generalized under one series.

The project was chosen because it combines numerical approximation with an intuitive demonstration of Fourier decomposition. I also chose this project for personal reasons, as Fourier analysis and holomorphic dynamics (the topic of my original project idea before settling on this one) are perhaps the two most profound and inspirational concepts that ultimately put me on the math and engineering path I am currently perusing.

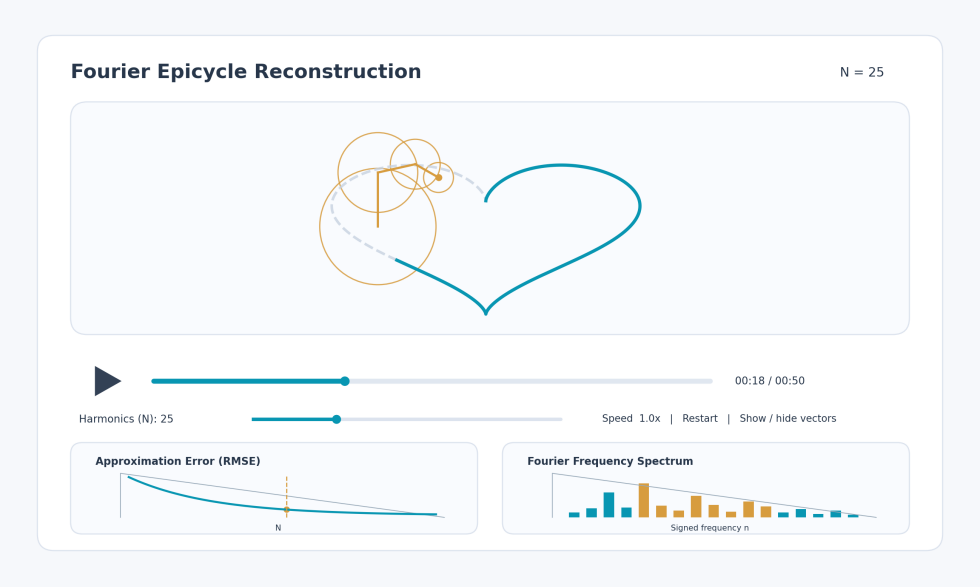


Figure 1: Conceptual visualization window. The main panel shows the epicycle reconstruction, with animation controls and a resolution slider. Side-by-side plots below show reconstruction error and the complex Fourier coefficient spectrum. The illustration is a GUI mockup (openAI), not computed output.

[1] Specifically, the normalization process will involve converting the raw mouse input data to an array that, while still technically *ordered* with respect to time, represents a continuous sample density across the entire curve.

[2] In particular, the video is entitled "But what is a Fourier series? From heat flow to drawing with circles", much of the src for which can be found on Grant's GitHub here.

Applied Methods

A captured curve will be represented by points in the complex plane, $z = x + iy$. The program will interpolate and resample the mouse input at approximately uniform arc-length intervals, obtaining M samples z_k with parameter values $t_k = k/M$. The sampled curve will then be approximated by the truncated complex Fourier series

$$z_N(t) = \sum_{n=-N}^{N} c_n e^{2\pi i n t}, \qquad c_n \approx \frac{1}{M} \sum_{k=0}^{M-1} z_k e^{-2\pi i n k/M}.$$

The coefficient calculation is a discrete Fourier transform (DFT), which numerically approximates the continuous Fourier coefficient integral. The project will implement the direct DFT to demonstrate the numerical method explicitly. Each term in $z_N(t)$ is a rotating vector: $|c_n|$ gives its radius, $\arg(c_n)$ its initial phase, and n its signed rotation frequency. Connecting these vectors head-to-tail yields the single epicycle chain; its final endpoint traces the reconstructed curve.

For quantitative evaluation, the program will compute the root mean square reconstruction error over the sampled parameter values:

$$\mathrm{RMSE}(N) = \sqrt{\frac{1}{M} \sum_{k=0}^{M-1} |z_k - z_N(t_k)|^2}.$$

This error will be plotted as a function of N. A companion frequency-spectrum subplot will display the magnitudes $|c_n|$, including both positive and negative frequencies. Since a Fourier series is periodic, open strokes with distinct endpoints will be treated as periodic paths with a connection from the last point back to the first; this limitation will be documented in the application.

Project Goals

- Allow the user to draw a continuous curve on a blank canvas or select a built-in preset curve.
- Convert the chosen path into a parameterized, uniformly resampled complex-valued curve and calculate its Fourier coefficients numerically.
- Reconstruct the drawing in a separate visualization using a *single chain* of epicycles, rather than independent horizontal and vertical Fourier animations.
- Provide a slider controlling the maximum harmonic index N, where the approximation uses frequencies $-N$ through N (up to $2N + 1$ terms).
- Include play/pause, restart, adjustable playback speed, and a timeline scrubber to inspect the epicycle chain at any point in the animation.
- Allow the user to show or hide the rotating circles/vectors and the original input curve for comparison.
- Place two side-by-side subplots beneath the main animation: one showing RMSE versus N, with the current setting marked, and the other showing the complex Fourier coefficient magnitudes, highlighting the included frequencies.

Requirements

The completed project will be evaluated against the following measurable specifications:

1. The application must accept a user-drawn continuous stroke containing at least 100 captured points and offer at least three selectable preset curves.
2. The application must interpolate and resample every accepted path into $M = 512$ points at approximately uniform arc-length intervals.
3. The application must compute complex Fourier coefficients using a directly implemented DFT and verify them against analytically known coefficients for at least two simple preset curves.
4. The reconstruction must use one complex-valued sum with frequencies $-N, \ldots, N$ and display the corresponding epicycle chain and its traced endpoint.
5. The user must be able to select any integer N from 0 through 100 using a slider, and the displayed reconstruction must update to match the selected value.
6. The program must provide functional play/pause, restart, speed adjustment, timeline scrubbing, and visibility controls for epicycles and the original curve.
7. The application must calculate and display RMSE for each integer N from 0 through 100, and mark the currently selected N on the error plot.
8. A second subplot beside the error plot must show $|c_n|$ for positive and negative frequencies and visually distinguish the coefficients included in the current reconstruction.
9. The project source code, usage instructions, and documented verification tests must be published in a public GitHub repository.